MACG 松尾　2019/2/20

JINJA2 & META PROGRAMING

このスライドのゴール

- 目的
  - メタプログラミングの意義とやり方を知る
- 対象
  - Xcodeを使っている
  - ある程度のプログラム知識
    - ジェネリクス（Template）を知っている
- 説明しないこと
  - Pythonに関すること。知らなくてもたぶん大丈夫。使う時に調べてください

メタプログラミングって何？

メタプログラミングって何？

A.　ソースコードを生成するためのプログラミング

メタプログラミングって何のためにつかうの？

- ソースコードをまとめて簡潔にするため
- Don’t Repeat Yourself

メタプログラミングって何？

- ジェネリクス（テンプレート）
  - 言語仕様として提供されているメタプログラミング手法
    - 標準的なので書きやすい
    - エラーもわかりやすい

ジェネリクスは超オススメ

ジェネリクスで対応できない場合がある

```
switch enumX_value {
  case enumX_a: return “a”;
  case enumX_b: return “b”;
  case enumX_c: return “c”;
}
```

うっかり書き間違えても気付きにくい

メタプログラミングって何？

- マクロ（C言語マクロ）
  - ジェネリクスが使用できない場合でも使用可能
  - 言語仕様として提供されているメタプログラミング手法
    - ☠️書きにくい＆読みにくい☠️
    - ☠️エラーの発生源を追うのが大変☠️

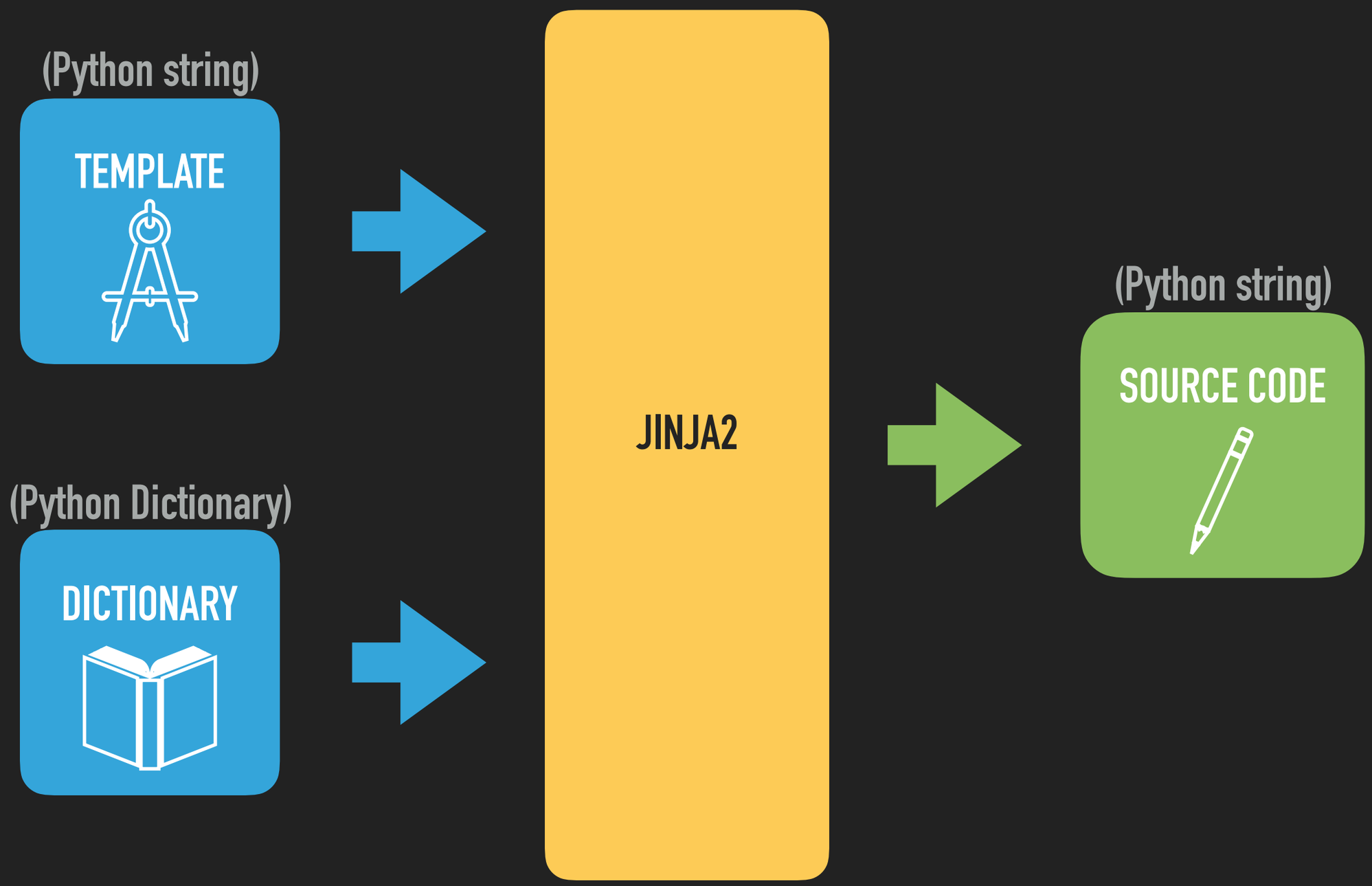
マクロは書きたくない！

PYTHON & JINJA2

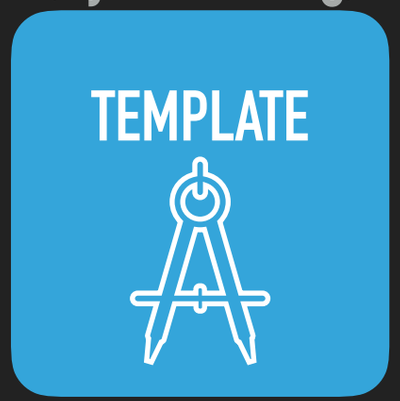
- Jinja2はPythonのライブラリ(http://jinja.pocoo.org/docs/2.10/)
  - 言語仕様ではないメタプログラミング
  - マクロよりも表現力が高い
  - Python スクリプトなので Xcode で実行可能
  - エラーわかりやすい

ジェネリクスで対応できないならオススメ

JINJA2


(Python string)
TEMPLATE
(Python Dictionary)
DICTIONARY
JINJA2
(Python string)
SOURCE CODE

(Python string)

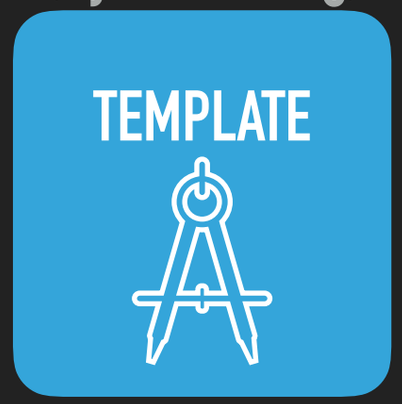



```
template = '''\
struct {{ name }} {
    {{ type }} {{ argu }};
    {{ type }} * pointer_{{ argu }};
};
'''
```

(Python Dictionary)

```
{“name”:”Apple”, “type”:”int”, “argu”:”seed”}
```

(Python string)



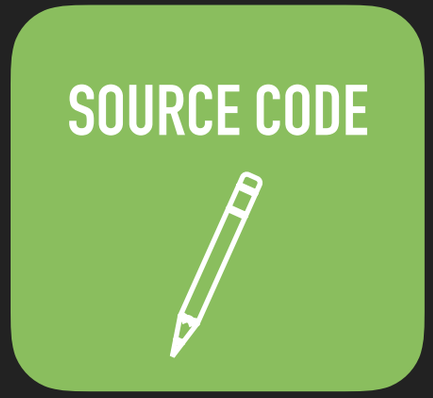

```
template = '''\
struct {{ name }} {
    {{ type }} {{ argu }};
    {{ type }} * pointer_{{ argu }};
};
'''
```

JINJA2

(Python Dictionary)

```
{"name":"Apple", "type":"int", "argu":"seed"}
```

PYTHON & JINJA2




```
struct Apple {
    int seed;
    int * pointer_seed;
};
```

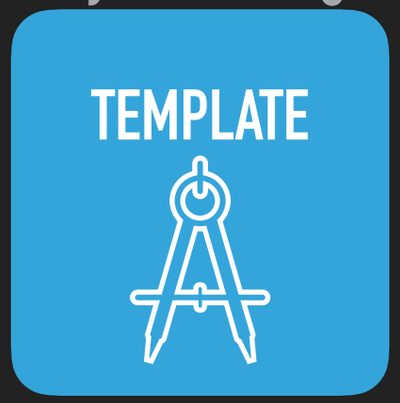
JINJA2でできること

- 変数の当てはめ (KeyPath的な書き方が可能)
- Python関数の呼び出し (例：SnakeCase 変換 CamelCase変換)
- For文、IF文
- マクロ（関数、変数のエイリアス）定義

JINJA2で生成してみよう

```
switch enumX_value {
  case enumX_a: return “a”;
  case enumX_b: return “b”;
  case enumX_c: return “c”;
}
```

(Python string)




```
template = '''\
switch enumX_value {
{% for e in enums %}
  case enumX_{{ e }}: return “{{ e }}”;
{% endfor %}
}
'''
```

（for文で繰り返す）

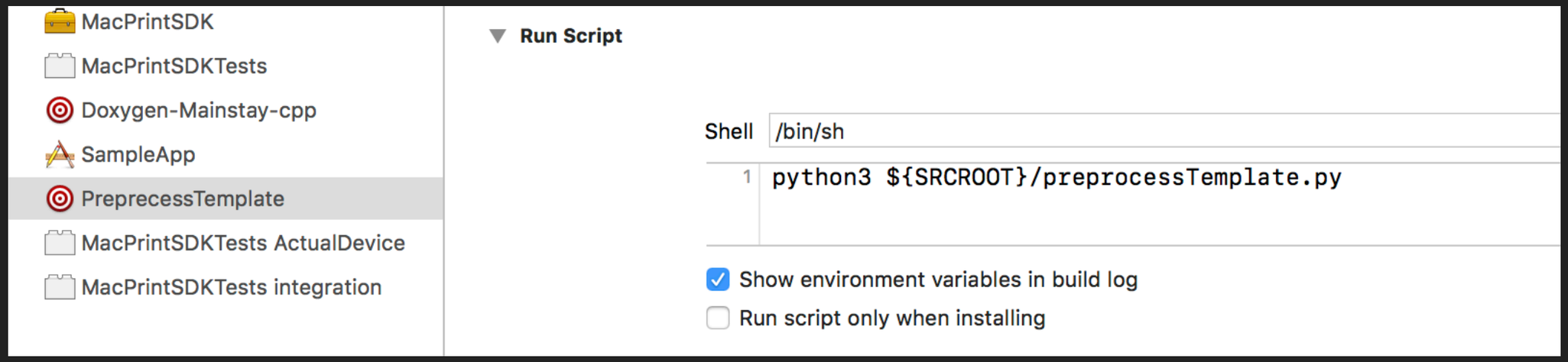
(Python Dictionary)

```
{“enums”:[“a”,”b”,”c”]}
```

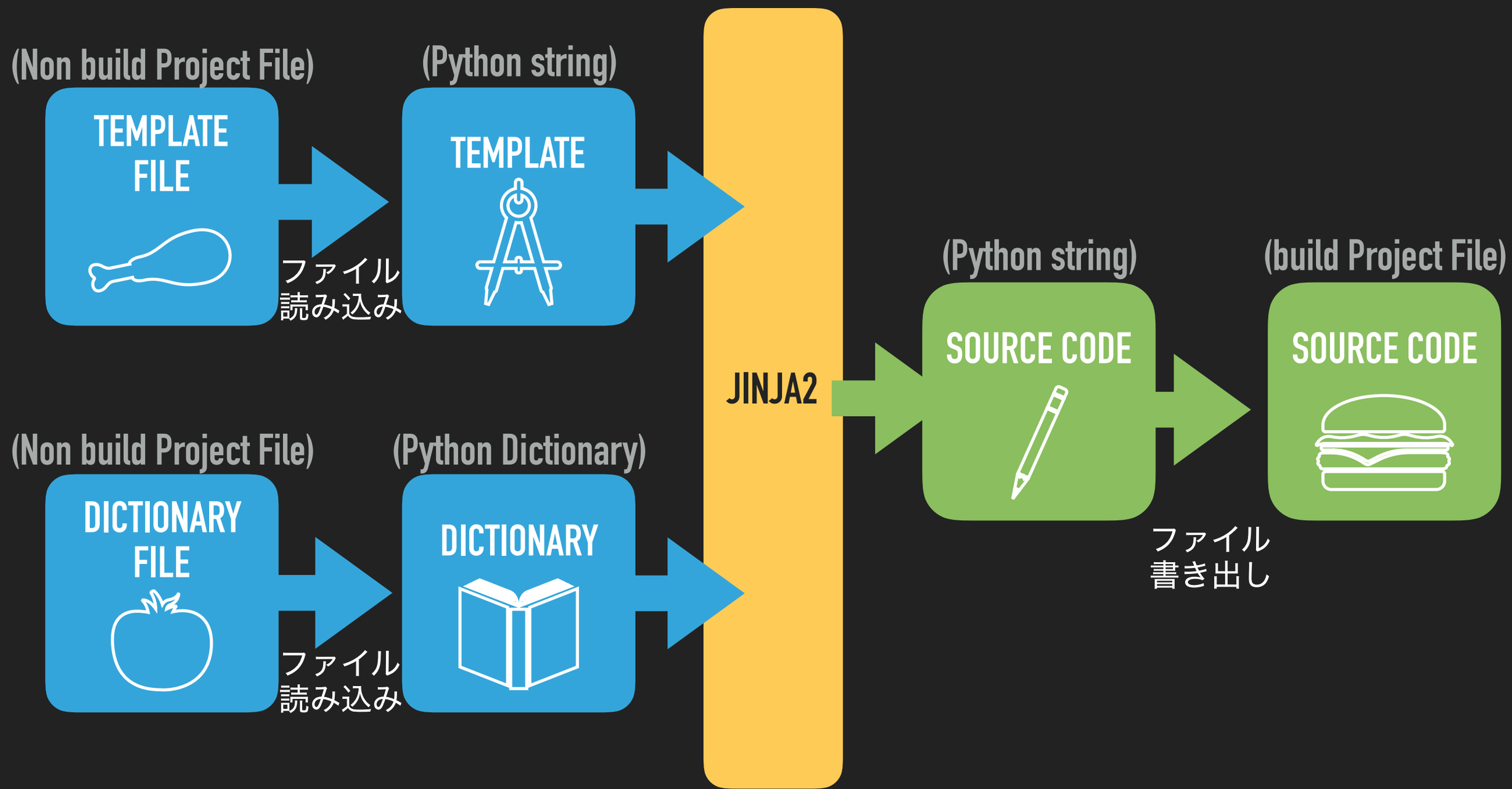
（配列）

XCODEに組み込むには

- Python3, Jinja2をインストール
- Xcodeに新しいTargetを作成してRun Scriptを追加
- Python3スクリプトを実行するように設定



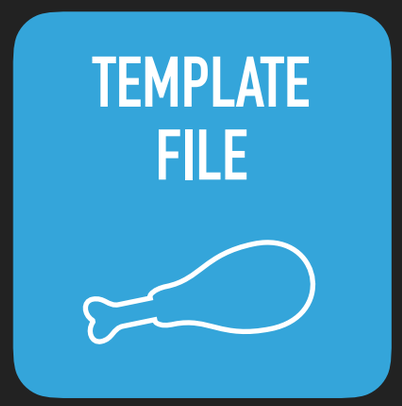
PYTHONスクリプトの処理の流れ



実用例

- BFElement
  - Mac ES SDKに使用する定数を定義するリポジトリ
  - C++, Objective-CのTemplateが作成済み
  - Template: 20KB, Dictionary:30KB -> Source: 745KB生成

BFELEMENTの解説



（Jinja2テンプレート形式）

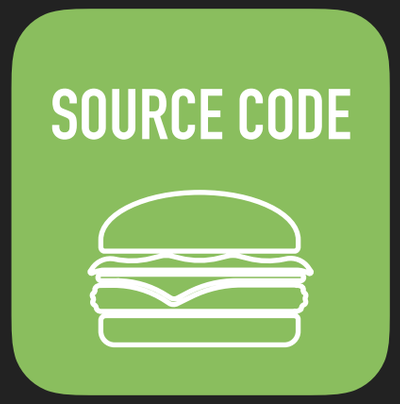
- 定数定義式のテンプレート
- 関数のテンプレート
- テストのテンプレート

（INIファイル形式）

- 定数のテンプレート
  - 名前
  - 型
  - 値

Pythonで簡単に読み込めるのでINIファイルがオススメ

XCODEに組み込むには



- Enum定義（C++, Objective-C）
- String定義（C++, Objective-C）
- Enum->String変換関数（C++, Objective-C）
- String->Enum変換関数（C++, Objective-C）
- C++Enum <-> Objctive-C Enum [Toll Free Bridge機能] のテスト
- C++ String <-> Objctive-C String [同一Byte保証機能] のテスト
- その他

XCODEに組み込むには

Demo

まとめ

- メタプログラミングにはジェネリクス（Template）を使う
- ジェネリクスが使えない時はJinja2を使うのがオススメ
  - Python3とJinjaをインストールすれば簡単に使える
- みんなBFElementを使ってね

付録 -BFELEMENTの詳しい解説-

- ファイル構成
  - ファイルの解説
    - 構成要素の解説
- 使い方
- Templateファイル
  - 使用できる変数の解説
  - 使用できるユーティリティ関数の解説

ファイル構成（ORIGIN/FEATURE/QL600700 JANUARY 8, 2019 [0F36884]時点）

- main.py - BFElementのJinja2ビルド全体を実行するpythonファイル
- output - Jinja2ビルド成果物（ソースコード）が格納されるフォルダ
- src - Jinja2ビルドのデータソースが格納されているフォルダ
  - create_definition.py - Jinja2ビルドに使用する自作ライブラリ
  - database - Dictionary Fileが格納されているフォルダ
    - element - Element Fileが格納されているフォルダ
    - element_set - ElementSet Fileが格納されているフォルダ
  - template - Jinja2のTemplateが格納されているフォルダ

FILEの解説 -ELEMENT FILE-

- 定数の1要素を定義するファイル、INIファイル形式
  - 1要素につき1ファイル

```
[media]
key: MediaSize
# type( enum, int, float, bool, string )
type: enum
values:
  L3.5mm
  L6mm
```

Section - Elementのカテゴリ分けに使用する

key - Elementの名前

type - Elementの型

values - (type:enumの場合のみ使用) Elementの取りうる値

FILEの解説 -ELEMENTSET FILE-

- 定数のセット(集まり)としての要素を定義するファイル、INIファイル形式

```
[print_settings] Section - ElementSetのカテゴリ分けに使用する
meta: PrintSettings  meta - ElementSetの名前
keys: keys - Elementの集まり (Elementのkeyの羅列)
# 用紙長さ ( strPaperName )
  MediaSize
# 画像印刷時の用紙の向き ( Orientation orient
  ImageOrientation
```

FILEの解説 -ELEMENTSET FILE-

- keysには以下の表現が使用可能
  - KeyName[]
    - KeyNameの配列
  - KeyName{SubKeyName}[]
    - (SubKeyName, KeyNameの配列) のDictionary

使い方 - 新しいELEMENTを作りたい

- src/database/elementに新しいiniファイルを追加する
  - Section - カテゴリごとにファイル分けされているのでカテゴリを新しく作ると新しい成果物ファイルが作られる
  - key - 固有の名前をつける。sectionが違っても名前空間は同じなので重複がないように注意する
  - type - (enum, int, float, bool, string)から適切なものを選択
  - values - enumの場合必要。プログラムによってvalue:__Nullが必ず追加される
    - valueは名前空間が異なるのでほかのElementのvalueと重複しても良い
      - 可読性のために過不足のない適切な名前をつけること

使い方 - 自動ソースコード生成

- 新しいpythonファイルを作成し、ビルド時に実行する

```
import os
from stat import *
rootDir = (os.path.dirname(os.path.abspath(__file__))) + "/"
bfElementDir = rootDir + “「BFElementの相対パス」”

import sys
sys.path.append(bfElementDir + "src")
import create_definition

elementDirName = "element/"
elementSetDirName = "element_set/"
databaseDir = bfElementDir + "src/database/"
elementDir = databaseDir + elementDirName
elementSetDir = databaseDir + elementSetDirName

create_definition.makeElementSetSource(elementSetDir,
                                       elementDir,
                                       「テンプレートのパス」,
                                       「出力先のパス」)
```

使い方 - 自動ソースコード生成

- Templateファイルで使用できる変数
  - mainNamespace : “bf”が入っている
  - subNamespaceElement : “element”が入っている
  - subNamespaceElementSet “element_set”が入っている
  - stringClass : “String”が入っている
  - enumClass : “Number”が入っている
  - elementDictionary : (meta, Element)のDictionary
    - Element : エレメントクラス
      - key
      - type
      - values
  - elementSetDictionary : (meta, elementSet)のDictionary
    - ElementSet
      - meta
      - keys
      - elements : Element(またはArrayOfElementまたはDictionaryOfArrayOfElementの混合)配列

使い方 - 自動ソースコード生成

- 補足（ElementSetに以下のKeyを使用したとき）
  - KeyName[] → ArrayOfElement
  - KeyName{SubKeyName}[] → DictionaryOfArrayOfElement

- ArrayOfElement : あるElementのvalueが複数の値をとることを表したクラス。Elementとインターフェイスの互換性がある
  - key : elementのkeyと同じ
  - type : Type.array が格納されている
  - element : そのElement
- DictionaryOfArrayOfElement :(extra_key_elementのvalue, key_elementのvalueの配列) のDictionaryの形式をとることを表したクラス。Elementとインターフェイスの互換性がある
  - key : key_elementのkeyと同じ
  - type : Type.dict_of_array が格納されている
  - key_element : そのElement
  - extra_key_element : DictionaryのKeyになるElement

TEMPLATE FILE内で使用可能なUTILITY関数解説

- inSyntax関数 - c構文中に使用できない文字を"_"に変換する
- snakeCase関数 - camel case文字列をsnake caseに変換する
- camelCase関数 - snake case文字列をcamel caseに変換する
- headLower関数 - 先頭の文字を小文字に変換する

- 詳しく知りたい場合はcreate_definition.pyを参照

TEMPLATE FILE内で使用可能な組み込み関数解説

- upper関数 - 全て大文字にする
- title関数 - 単語の先頭の一文字を大文字、他を小文字にする
- replace関数 - 文字を他の文字に変換する
- ...等

- stringのメンバ関数は全て使用可能